

C++&Qt Day4 hw2 보고서

통신(UDP)의 이해/과제1 보고서

휴머노이드 수습단원 이주원

목차

1. 통신 이란?

1.1 통신방식

1.2 회선 구성 방식

1.2.1 패킷 교환 방식

1.3 통신 프로토콜

2. UDP

2.1 TCP/IP 4계층과 전송 계층

2.2 UDP란?

2.3 UDP 헤더 구조

2.4 UDP 동작

2.5 UDP 용도

2.6 포트번호

2.7 소켓 주소

2.8 장점과 단점

3. 과제 1번 보고서

1. 통신이란?

- 어떤 정보를 다른 상대방에게 전달하여 시간과 거리의 제한을 극복하는 과정
- 송신자->수신자 정보전달 행위
- 구성요소: 송신자, 수신자, 통신 매체, 프로토콜

1.1 통신방식

- 단방향
- 반이중
- 전이중, 양방향

	단방향	반이중	전이중
통신방향	단방향 (Unidirectional)	쌍방향 (Two_directional, One by One)	양방향 (Bi-directional, Simultaneously)
Sender-Receiver	Sender는 데이터만 전송할 수 있다	Sender는 데이터를 하나씩 주고 받을 수 있다	Sender는 데이터를 동시에 주고 받을 수 있다
성능	나쁨	보통	좋음

- 단방향 통신

단방향 전송모드에서 Sender와 Receiver 사이의 통신은 '한 방향'으로만 이루어진다. Sender는 오직 데이터를 전송할 수만 있고, Receiver는 오직 데이터를 수신할 수만 있다. Receiver는 Sender에게 응답하지 못한다

키보드와 모니터와 같은 단순 입출력 장치들이 단방향 통신장치에 해당한다. 리모컨에서 TV로 신호가 갈 뿐이며, 반대 방향으로의 신호 전송은 없다. 살짝 이야기가 겹쳐서 셀 수 있겠지만, TV, 라디오 방송도 단방향 통신이다. 과거에는 방송과 통신이 엄격히 구분이 되었으나, 오늘날에는 방송과 통신의 구별은 모호해 졌기 때문에 방송을 단방향 통신의 예로 들어 보았다. 한편, 전화나 컴퓨터 통신의 경우에는 신호가 연결된 두 단말기기의 양방향으로 전송이 된다

- 반이중 통신

반이중 전송모드에서 Sender와 Receiver 사이의 통신은 한번에 하나씩이지만 '쌍방향'으로 이루어진다. Sender와 Receiver는 정해진 시간 안에서 정보를 전송하고 수신할 수 있다

무전기를 사용하는 것처럼 한 사람씩 이야기하고 듣는 통신방식이다. 한쪽이 통화 버튼 (Push-To-Talk)을 누르고 말을 하게 되면, 상대방은 듣기만 할 수 있으며, "오버"라는 말과 함께 자신의 차례가 끝났음을 알리면, 이제는 역할을 바꾸게 된다

- 전이중 통신

전이중 통신에서 Sender와 Receiver 사이의 통신은 '동시에' 일어난다. Sender와 Receiver는 동시에 정보를 전송하고 수신할 수 있다

일반 전화기로 대화를 하는 것처럼 언제든지 송신과 수신이 동시에 가능한 경우이다. 이더넷 통신을 비롯해서 오늘날 데이터 통신은 송수신이 동시에 가능한 전이중 통신이다.

전이중 통신은 일반적으로 전송을 위한 라인과 수신을 위한 라인이 각각 별도로 존재한다. 그러나 하나의 라인을 사용해서도 전이중 통신이 가능하다. 이를 위한 몇 가지 기법들이 사용되고 있다.

a. 시분할 이중통신(TDD, Time Division Duplex)

- 시간을 나누어 일정 타임 슬롯 주기로 송수신 방향 결정

-완벽한 전이중 통신 아님

-인지 범위를 넘어선 빠른 통신으로 전이중 통신으로 느껴짐

-멀티 태스킹과 같이, 실제로는 순차적으로 처리가 되고 있지만, 매우 빠른 속도로 문맥 교환이 이루어지므로 마치 병렬로 동시에 처리되는 것처럼 보이는 것과 같은 원리

-TD-CDMA, 와이 파이, 블루투스 등에 사용

- 주파수 분할 이중통신(FDD, Frequency Division Duplex)

-통신하는 주파수 대역을 분할하여 전송되는 데이터가 충돌되지 않도록 하는 방법

-셀룰러 이동 통신, ADSL, VDSL

1.2 회선 구성 방식

회선 교환 방식

- 2개의 컴퓨터 단말기 간에 정해진 통신로나 회선을 설정하여 정보를 교환하는 방식
- 음성 전화 시스템에 사용
- 일대일로 데이터 교환, 전화 간 통화 중에는 다른 상대와 통화 불가능

특징

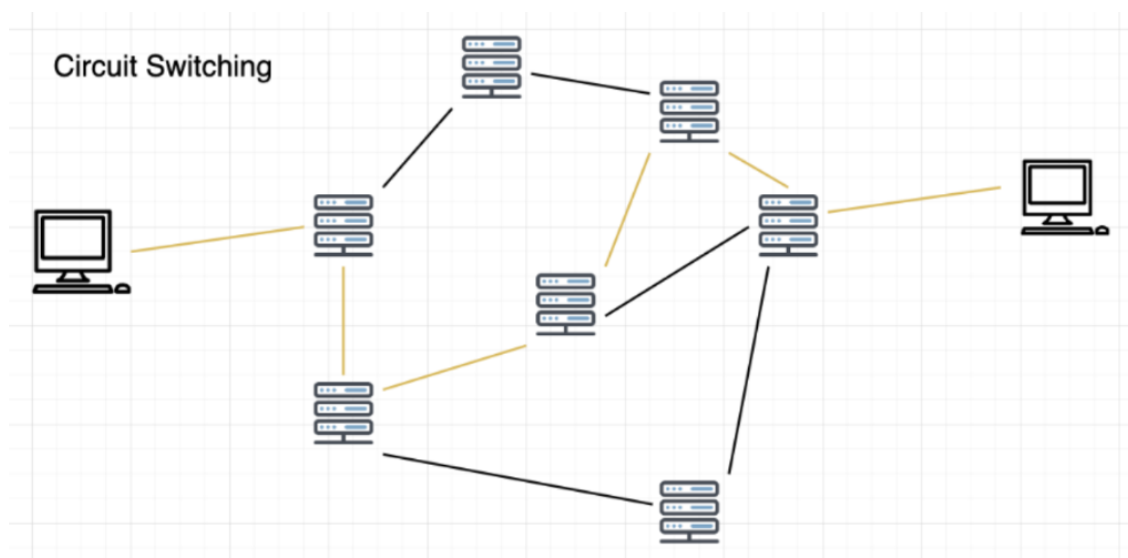
- 통신로가 설정->해당 통신로 전용 회선으로 사용 가능
- 부분망은 통신로를 통하는 정보 간섭X -> 정보의 형태, 부호, 전송 제어 절차 등에 제약X
- 대량의 정보를 송신하는 팩스, 통신 밀도가 높은 데이터 통신에 적합

장점

- 설정된 통신은 안정적
- 다른 요인에 의한 통신 방해X
- 연속적이고 대용량 데이터 처리에 용이

단점

- 통신연결 언제나 보장X
- 회선 이용 효율 떨어짐(대역폭 낭비)
- 통신 중 회선 문제 발생 시 연결 끊어짐+회선할당부터 다시



패킷 교환 방식

- 작은 블록의 패킷으로 데이터를 전송하여 데이터를 전송하는 동안만 네트워크 자원을 사용하도록 하는 방법
- 패킷은 여러 통신 지점(Node)을 연결하는 데이터 연결 상의 모든 노드들 사이에 개별적으로 경로가 제어된다.
- 통신 기간 동안 독점적인 사용을 위한 작은 데이터 트레픽에 적합

특징

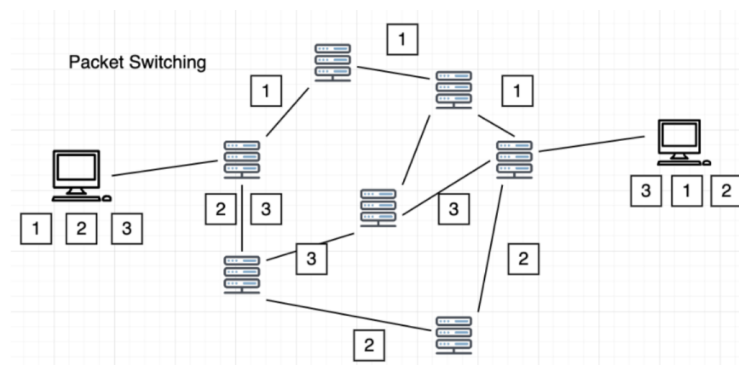
- 여러 경로 이용 가능, 가장 효율적인 경로 탐색
- 전송 속도 및 흐름 제어 가능
- 송신 패킷 순서, 수신 패킷 순서 다를수 있음
- 에러탐지 가능
- 일반적인 인터넷 망에서 사용

장점

- 회선 효율성 높음
- Store_and_Forward 방식을 사용하여 각 스테이션에 맞는 속도 조절 가능
- 상황에 따른 교환기 및 회선 등에 장애가 발생하여도 패킷 우회전송으로 전송신뢰성 증가
- 전송 지연 감소 및 안정성 증가

단점

- 데이터 단위 길이 제한
- 경로 중 각 교환기에서 다소의 지연 발생
- 패킷 교환을 위한 소프트웨어 및 하드웨어 복잡
- 각 패킷마다 주소를 위한 오버헤드 존재



1.2.1 패킷 교환 방식

데이터 교환 방식

- 비연결 지향적 성격
- 관련된 패킷이라도 따로 전송
- 패킷의 경로가 다를수 있고 망의 상황에 따라 변하며 패킷의 도착 순서 바뀔 수 있음
- 순서의 재조정 필요

장점

- Call Setup 과정 불필요 -> 소수 패킷만 보낼 때 빠르고 오버헤드 적음
- 망 자원이 부족할 경우 다른 경로 대체 -> 망 운용의 유통성
- 망 고장일 때 최적의 경로 -> 신뢰성 증가

가상 회선 방식

- 회선 교환 방식 차용
- 연결 지향적인 특성
- 전송하기 전 논리적으로 목적지와 연결됨.
- 각 패킷당 가상 회선 식별 번호 존재
- 식별 번호를 통한 순서에 맞는 목적지 도착
- 관련된 패킷은 전부 같은 경로 이용

장점

- 각 노드에서 처리시간 적게 소모
- 패킷의 출발 순서대로 도착하여 오류 제어 용이
- 전송하는 데이터가 많아도 Call Setup을 한 번만 하면됨 -> 효율적인 전송
- 데이터 전송 전 Call Setup이 이루어져 라우팅 테이블 등록 -> 회선을 전용하지 않아 각
각의 패킷에 대한 각각의 경로 지정 필요X

단점

- Call Setup에 대한 오버헤드 존재

1.3 통신 프로토콜

통신 프로토콜(Communication Protocol)은 서로 다른 기기들 간의 데이터 교환을 원활하게 수행할 수 있도록 표준화시켜 놓은 통신 규약. 통신을 제어하기 위한 표준 규칙과 절차의 집합으로 하드웨어와 소프트웨어, 문서를 모두 규정하는 말.

기본요소

구문(Syntax) : 전송하고자 하는 데이터의 형식, 부호화, 신호 레벨등을 규정

의미(Semantics) : 두 기기 간의 효율적이고 정확한 정보 전송을 위한 협조 사항과 오류 관리를 위한 제어 정보를 규정

시간(Timing) : 두 기기 간의 통신 속도, 메시지의 순서 제어 등을 규정



- 시스템(System) : 컴퓨터, 터미널 등
- 개체(Entity) : 사용자 프로그램, 파일 전송 프로그램, 데이터 베이스 등

2. UDP

2.1 TCP/IP 4계층과 전송 계층

UDP를 이해하려면 먼저 UDP가 어느 위치에 있는 프로토콜인지 알아야 한다.

계층	이름	역할	대표 프로토콜
4	응용 계층 (Application)	사용자 프로그램과 직접 맞는 계층	HTTP, FTP, DNS, DHCP
3	전송 계층 (Transport)	종단 간(end-to-end) 데이터 전달, 포트로 프로세스 구분	TCP, UDP
2	인터넷 계층 (Internet)	호스트 간 경로 결정, 주소 지정	IP, ICMP, ARP
1	네트워크 접속 계층 (Network Access)	물리적 전송, 프레임 단위 처리	Ethernet, Wi-Fi

-IP는 호스트(컴퓨터)까지 데이터를 전달한다.

-전송 계층은 그 호스트 안의 어떤 프로그램(프로세스)에게 전달할지를 결정한다. 이 구분에 사용되는 것이 포트 번호다.

-전송 계층에는 대표적으로 TCP와 UDP가 있고, 앞서 1.2에서 다룬 회선 교환 / 패킷 교환의 관점에서 보면 UDP는 비연결형 패킷 교환(데이터그램 방식)에 해당한다.

2.2 UDP란?

UDP(User Datagram Protocol, 사용자 데이터그램 프로토콜)는 전송 계층에서 동작하는 비연결형(connectionless) 프로토콜이다. RFC 768에 정의되어 있으며, 문서 자체가 3페이지밖에 되지 않을 만큼 매우 단순한 프로토콜이다.

핵심 특징

- 비연결형: 데이터를 보내기 전에 연결을 설정하는 과정(TCP의 3-way handshake)이 없다. 보내고 싶을 때 그냥 보낸다.
- 신뢰성 보장 없음(Unreliable): 패킷이 도착했는지 확인(ACK)하지 않는다. 손실되어도 재전송하지 않는다.
- 순서 보장 없음: 먼저 보낸 패킷이 나중에 도착할 수 있다. 순서 재조립을 해주지 않는다.
- 흐름 제어 / 혼잡 제어 없음: 수신 측이 처리할 수 있든 없든 송신 측 속도대로 계속 보낸다. 네트워크가 혼잡해도 전송 속도를 줄이지 않는다.
- 데이터그램(Datagram) 단위 전송: 경계(boundary)가 보존된다. 송신 측에서 1024바이트를 한 번 보내면 수신 측도 1024바이트를 한 번에 받는다. (TCP는 바이트 스트림이라 경계가 없다)
- 헤더가 매우 작음: 8바이트. (TCP는 최소 20바이트)
- 오버헤드가 적고 빠름: 위의 기능들을 전부 생략했기 때문에 지연이 낮고 처리 부담이 적다.

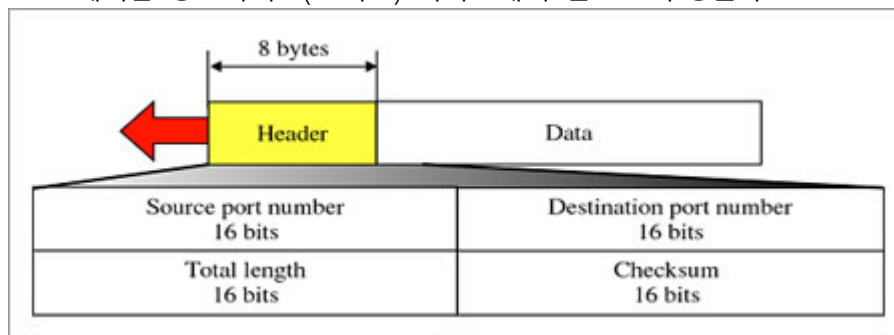
신뢰성이 없다는 말의 의미

UDP가 신뢰성이 없다는 것은 UDP 자체가 아무 보장도 하지 않는다는 뜻이지, 통신이 항상 실패한다는 뜻이 아니다. 실제로는 로컬 네트워크(유선 LAN)에서 UDP 패킷이 유실되는 경우는 거의 없다. 다만 프로토콜 차원에서 보장해 주지 않기 때문에, 신뢰성이 필요하면 응용 계층에서 직접 구현해야 한다.

- 순번(sequence number)을 데이터에 직접 넣어 순서 확인
- ACK 메시지를 응용 계층에서 주고받아 수신 확인
- 타임아웃 후 재전송 로직 직접 구현
- CRC 등 추가 검증 코드 삽입

2.3 UDP 헤더 구조

UDP 헤더는 총 8바이트(64비트)이며 4개의 필드로 구성된다.

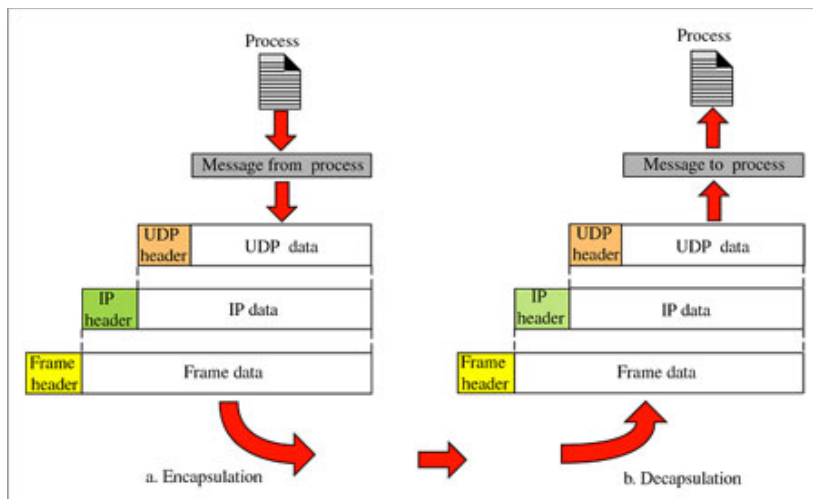


- 발신지 포트 번호 (Source Port Number)
 - = 발신지 호스트 상에서 수행되는 프로세스가 사용하는 포트 번호이다.
- 목적지 포트 번호 (Destination Port Number)
 - = 목적지 호스트 상에서 수행되는 프로세스가 사용하는 포트 번호이다.

- 길이 (Length)
 - = 헤더와 데이터를 합한 사용자 데이터그램의 전체 길이를 정의한다.
- 검사합 (Checksum)
 - = 사용자 데이터그램에 오류가 있는지 검사하기 위해 사용한다.
 - = 12Byte의 의사 헤더(Pseudo Header)를 추가하여 IP 헤더의 오류로 잘못된 호스트에게 전달되는 것을 방지한다.

2.4 UDP 동작

- 비 연결형 서비스
 - = UDP는 비 연결형(Connectionless) 서비스를 제공한다.
 - = UDP에 의해 보내어진 각 사용자 데이터그램은 서로 독립적이다.
 - = 동일한 발신지 프로그램으로 부터 동일한 목적지 프로그램으로 전송되더라도 서로 다른 데이터그램 사이에는 아무런 연관 관계가 없다.
 - = UDP는 프로세스가 긴 데이터 스트림을 보냈을 경우 이 스트림을 서로 연관된 사용자 데이터그램들로 분할하지 않는다.
 - = 하나의 사용자 데이터그램에 들어갈 수 있는 작은 메시지를 보내는 프로세스만 UDP를 사용할 수 있다.



- 캡슐화와 역캡슐화
 - = 캡슐화 (Encapsulation)
 - ① 프로세스가 UDP를 통하여 전송할 메시지가 있을 때 UDP로 메시지와 한 쌍의 소켓 주소 그리고 데이터 길이를 보낸다.
 - ② UDP는 데이터를 받은 후 UDP 헤더를 추가한다.
 - ③ UDP는 소켓 주소들과 함께 사용자 데이터그램을 IP로 전달한다.
 - ④ IP는 자신의 헤더를 추가하는데 이 헤더에서 프로토콜 필드의 값을 17로 설정하여 데이터가 UDP로부터 왔음을 알리고 IP 데이터그램은 데이터 링크 계층에 전달된다.
 - ⑤ 데이터 링크 계층은 자신의 헤더를 추가하고 그 결과를 물리 계층으로 전달하고 물리 계층은 비트들을 전기 또는 광신호로 부호화한 후 이를 목적지로 전송을 하게 된다.

= 역캡슐화 (Decapsulation)

- ① 메시지가 목적지 호스트에 도착하면 물리 계층은 신호를 비트로 복호화한 후 데이터 링크 계층으로 전달한다.
- ② 데이터 링크 계층은 헤더와 트레일러를 사용하여 데이터를 점검한 후 오류가 없으면 헤더와 트레일러를 삭제하고 데이터를 IP로 전달한다.
- ③ IP 소프트웨어는 IP 단계의 검사를 수행한 후 오류가 없으면 헤더를 제거하고 송신자와 수신자 IP 주소와 함께 사용자 데이터그램을 UDP로 전달한다.
- ④ UDP는 검사합을 사용하여 전체 사용자 데이터그램을 검사한 후 오류가 없으면 헤더를 제거하고 송신자 소켓 주소와 함께 응용 데이터를 프로세스에 전달하고 수신된 메시지에 대해 응답을 보낼 필요가 있을 경우를 대비하여 송신자 소켓 주소가 프로세스에 전달하게 된다.

- 큐잉

= 클라이언트 프로세스

- ① UDP에서는 포트에 큐(Queue)가 연동
- ② 프로세스가 시작되면, 운영체제에게 포트번호 요청
- ③ 큐는 포트번호로 식별
- ④ 프로세스 종료 시 큐도 제거
- ⑤ 출력큐가 가득 차서, 큐에 넣을 수 없을 때 운영체제는 프로세스에게 대기 요청
* 소켓 프로그램에서 sendto()가 blocking 됨
- ⑥ 메시지가 클라이언트에 도착하면 UDP는 지정된 포트번호에 대한 입력큐 존재 여부 확인. 입력큐가 있으면 큐에 삽입
- ⑦ 입력큐가 없으면, 데이터그램 폐기한 후 ICMP에 "도달불가" 메시지 전달 요청

= 서버 프로세스

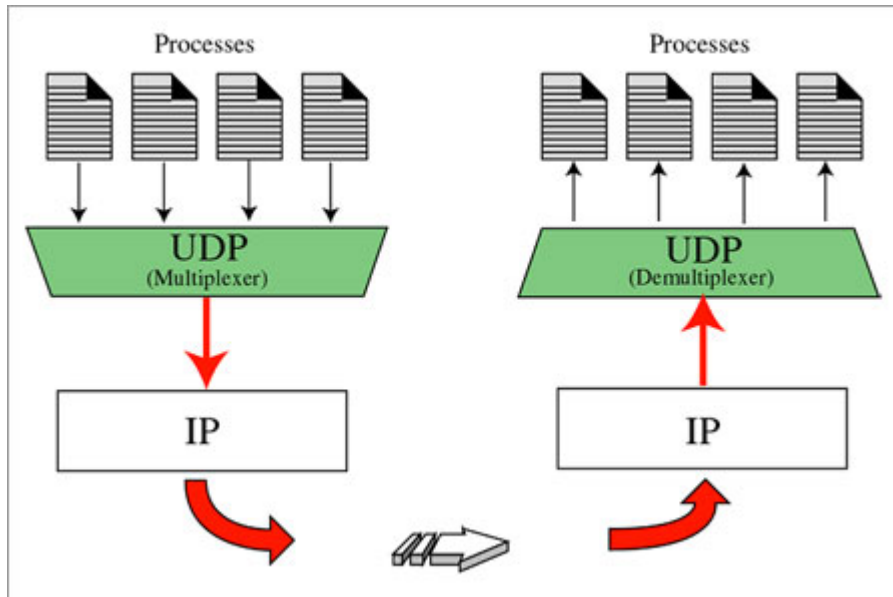
- ① 운영체제에 따라 큐 생성방법이 다름
- ② 서버 프로세스가 수행될 때, 잘 알려진 포트에 대하여 입력 큐와 출력 큐 생성
- ③ 서버에 메시지가 오면, 사용자 데이터그램의 목적지 포트번호로 지정된 포트에 입력 큐 존재 여부를 확인.
입력큐가 있으면 큐에 삽입
- ④ 입력큐가 없으면, 데이터그램 폐기한 후 ICMP에 "도달불가" 메시지 전달 요청
- ⑤ 입력큐에 오버플로우(Overflow)가 발생해도 데이터그램 폐기 후, ICMP에 "도달불가" 메시지 전달 요청
- ⑥ 서버에 클라이언트에 응답하기를 원하는 경우, 수신 데이터그램에 있는 발신지

포트번호를 이용해 생성된 데이터그램을

출력큐에 삽입

- ⑦ 출력큐가 가득 차서, 사용자 데이터그램을 넣지 못하면, 운영체제는 프로세스에게 대기 요청

* 소켓 프로그램에서 sendto()가 blocking 됨



- 다중화와 역다중화

= 다중화 (Multiplexing)

: 송신자 측에서는 사용자 데이터그램을 보내고자 하는 프로세스가 여러 개 있을 수 있으나 UDP는 하나만 있으므로 다대일 관계이기 때문에 다중화를 요구한다.

= 역다중화 (Demultiplexing)

: 수신자 측에서도 UDP는 하나이지만 데이터그램을 보내야 하는 프로세스는 여러 개가 있는 일대다 관계이기 때문에 역다중화를 필요로 한다.

2.5 UDP 용도

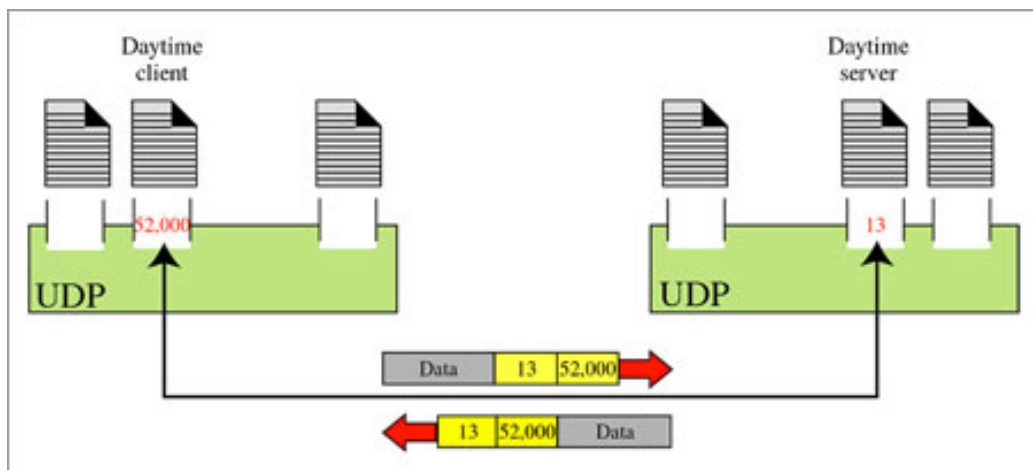
- 단순한 요청-응답을 필요로 하는 프로세스
- 흐름 제어와 오류 제어가 크게 필요하지 않은 프로세스
- 내부적인 흐름 제어와 오류 제어 매커니즘을 가지고 있는 프로세스
- 멀티캐스팅과 브로드캐스팅을 위한 전송 프로토콜
- SNMP(Simple Network Management Protocol)와 같은 관리 프로세스
- RIP(Routing Information Protocol)와 같은 경로 갱신 프로토콜에 사용

2.6 포트번호

프로세스 통신의 가장 보편적인 방법은 클라이언트 / 서버 모델이다.

이런 프로세스 대 프로세스를 정의하기 위한 식별자로 포트 번호가 사용된다.

- ① 클라이언트 / 서버 모델에서 통신을 위한 사항
 - = 로컬 호스트(Local Host)와 로컬 클라이언트 프로세스(Local Process)
 - = 원격 호스트(Remote Host)와 원격 서버 프로세스(Remote Process)
- ② 로컬 호스트와 원격 호스트는 IP 주소를 사용하여 정의한다.
- ③ 프로세스들을 정의하기 위하여 0 ~ 65,536 사이의 포트 번호를 사용한다.
- ④ 클라이언트 프로그램에서는 UDP 소프트웨어가 임의로 선택한 임시 포트 번호(Ephemeral Port Number)를 사용한다.
- ⑤ 서버를 위해서는 공인 포트 번호(Well-Known Port Number)를 사용한다.
- ⑥ 호스트가 선택된 후에는 포트 번호를 사용하여 특정 프로세스를 정의한다.



* 클라

이언트 (Client)

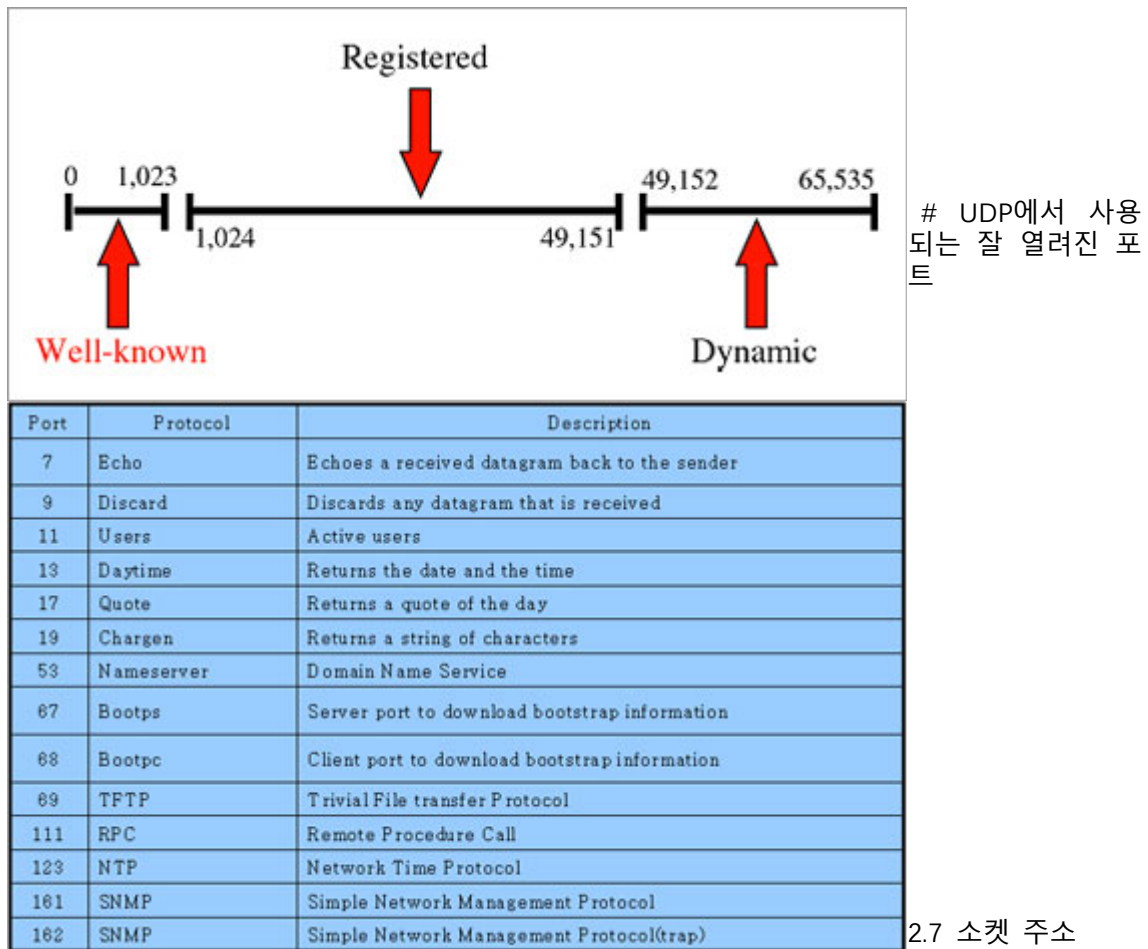
클라이언트는 다른 프로그램에게 서비스를 요청하는 프로그램

* 서버 (Server)

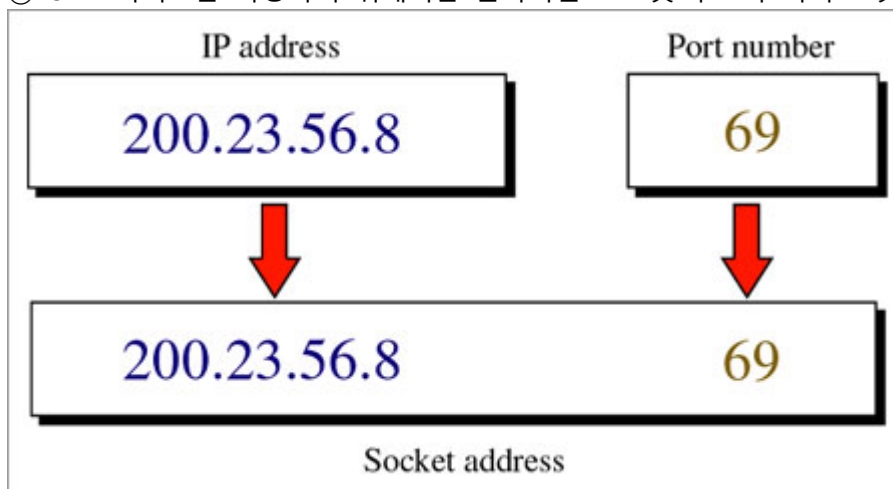
클라이언트 요청에 대해 응답을 해주는 프로그램

IANA (Internet Assigned Numbers Authority) 범위

- ① 잘 열려진 포트 : 0 ~ 1,023
- ② 등록된 포트 (Registered Port) : 1,024 ~ 49,151
- ③ 동적 포트 (Dynamic Port) : 49,152 ~ 65,535



- ① 소켓 주소는 IP 주소와 포트 번호를 조합한 것이다.
 클라이언트 소켓 주소는 클라이언트 프로세스를 유일하게 정의한다.
 서버 소켓 주소는 서버 프로세스를 유일하게 정의한다.
- ② UDP 서비스를 사용하기 위해서는 클라이언트 소켓 주소와 서버 소켓 주소가 필요하다.



3. 과제 보고서

파일구조

Project_name

```
|—— CMakeLists.txt
|—— include
|   |—— keypad.h
|   |—— mainwindow.h      //클래스 설계도
|—— resources
|—— src
|   |—— keypad.cpp
|   |—— main.cpp          //구현
|   |—— mainwindow.cpp
|—— ui
    |—— mainwindow.ui      //버튼등 화면 배치
```

=====

헤더파일

private slots:

```
void udp_read();
```

```
void udp_send();
```

시그널을 받아서 실행될 함수들이라 슬롯으로 선언했다. udp_read는 소켓의 readyRead, udp_send는 QLineEdit의 returnPressed에 연결된다. 두 시그널 모두 인자가 없기 때문에 슬롯도 인자를 가질 수 없다. 그래서 보낼 내용은 함수 안에서 ui->lineEdit->text()로 직접 읽어온다. 밖에서 호출할 일이 없어 private으로 두었다. connect는 private 슬롯이어도 정상적으로 연결된다.

private:

```
Ui::MainWindow *ui;
```

```
QUdpSocket *text_socket;
```

```
keypad *pad;
```

생성자가 끝난 뒤에도 계속 들고 있어야 하는 것들이라 멤버 변수로 선언했다. 특히 text_socket은 생성자에서 만들어 udp_send, udp_read 양쪽에서 써야 하므로 지역변수로 두면 안 된다. 셋 다 포인터로 선언하고 생성자에서 new로 만든다. text_socket과 pad는 부모를 this로 지정해서 만들었기 때문에 MainWindow가 소멸할 때 Qt가 자동으로 지워준다. ui만 QObject가 아니라 소멸자에서 직접 delete 해야 한다. pad는 이전 과제의 T9 키패드를 별도 클래스로 분리한 것이다. keypad가 QLineEdit에 글자를 써주면 기존 udp_send가 그대로 읽어가는 구조라, UDP 코드는 수정하지 않고 합칠 수 있었다.

=====

소스 파일

```
#define port1 50000
```

상대방과 포트 번호를 적을때 여기서 숫자만 바꿔도 할 수 있도록 해놨다.

```
if(text_socket->bind(QHostAddress::AnyIPv4, port1, QUdpSocket::ShareAddress))
{
    connect(text_socket, & QUdpSocket::readyRead,
this, &MainWindow::udp_read);
}
```

if문은 bind가 성공했을 때만 connect 한 것. 이 때만 패킷 오면 udp_read() 호출됨
*예제는 operator_ip를 사용했으나 여기서는 AnyIPv4를 사용했다. 이유는 와이파이를 옮기면
내 아이피가 바뀌는데 operatop_ip는 매번 그 때 마다 바꿔줘야한다. AnyIPv4는 안 해도 됨.

```
connect(ui->lineEdit, &QLineEdit::returnPressed, this, &MainWindow::udp_send);
lineEdit에 엔터가 눌러지면 udp_send()함수 호출
```

```
void MainWindow::udp_send(){
    QString msg = ui->lineEdit->text();
    text_socket->writeDatagram(msg.toUtf8(),QHostAddress("172.100.5.97"),port1);
    ui->textBrowser->append("나 : " + msg); ui->lineEdit->clear(); }
msg은 lineEdit에 적은 문자. 입력했던 문자를 textBrowser에서 "나: "+msg로 표현되게 설정
하였다.
```

```

void MainWindow::udp_read()
{
    while(text_socket->hasPendingDatagrams())
    {
        QNetworkDatagram datagram = text_socket->receiveDatagram();
        QString msg = QString::fromUtf8(datagram.data());
        ui->textBrowser->append(datagram.senderAddress().toString()+" : " +msg);
    }
}

```

UDP를 받아오는 함수. 상대방의 주소와 상대방이 보낸 메시지를 textBrowser에 한 번에 뜨게 하였다. 메시지를 utf8로 받아서 한국어도 오류 없이 받을 수 있도록 하였다.

실행화면

